

Support Non-copyable Types for `single_view`

Document #: P2483R0
Date: 2021-09-30
Project: Programming Language C++
Audience: LEWG
Reply-to: Hui Xie
<hui.xie1990@gmail.com>

§ 1 Introduction

This paper proposes allowing `single_view` to hold non-copyable types.

§ 2 Motivation and Scope

Currently, `single_view` requires the object that it holds to satisfy `std::copy_constructible` concept. This makes the `single_view` not usable with move only types. For example, the following code is invalid

```
// foo is move only type  
foo make_foo();  
  
std::views::single(make_foo()) // | more_pipe_lines
```

In the original c++ 20 ranges proposal, the view concept requires semiregular, thus all view implementations have to be copyable. However, after [P1456R1], view no longer requires implementations to be copyable but only movable, so technically the `std::copy_constructible` constraint can be relaxed to `std::move_constructible` for the types that `single_view` holds.

Before	After
<pre>// foo is move only type foo make_foo(); std::views::single(std::make_shared<foo> (make_foo())) std::views::transform([](const auto& f) -> decltype(auto) { return (*f); }) // more_pipe_lines</pre>	<pre>// foo is move only type foo make_foo(); std::views::single(make_foo()) // more_pipe_lines</pre>

§ 3 Impact on the Standard

This proposal is a pure library extension.

§ 4 Proposed Wording

Modify 24.2 [\[ranges.syn\]](#)

```
// [range.single], single view
template<copy_constructiblemove_constructible T>
    requires is_object_v<T>
class single_view;
```

Modify 24.6.3.2 [\[range.single.view\]](#)

```
namespace std::ranges {
    template<copy_constructiblemove_constructible T>
        requires is_object_v<T>
    class single_view : public view_interface<single_view<T>> {
    private:
        copyable-boxmovable-box<T> value_; // exposition only (see
            [range.copymove.wrap])

    public:
        single_view() requires default_initializable<T> = default;
        constexpr explicit single_view(const T& t) requires
            copy_constructible<T>;
        constexpr explicit single_view(T&& t);
```

```
constexpr explicit single_view(const T& t) requires copy_constructible<T>;
```

- 1 *Effects*: Initializes value_ with t.

Add a new section **[range.move.wrap]** under 24.7 [\[range.adaptors\]](#)

Movable wrapper

- 1 *movable-box* behaves exactly like `optional<T>` with the following differences:

(1.1) — *movable-box* constrains its type parameter T with `move_constructible<T> && is_object_v<T>`

(1.2) — The default constructor of *movable-box* is equivalent to:

```
constexpr movable-box()
    noexcept(is_nothrow_default_constructible_v<T>)
    requires default_initializable<T>
    : movable-box(){in_place}
    {}
```

(1.3) — If `movable<T>` is not modelled, the move assignment operator is equivalent to:

```
movable-box& operator=(movable-box&& that)
    noexcept(is_nothrow_move_constructible_v<T>) {
    if (this != addressof(that)) {
        if (that) emplace(std::move(*that));
        else reset();
    }
    return *this;
}
```

2 *Recommended practices: movable-box* should store only a T if either T models `movable` or `is_nothrow_move_constructible_v<T>` is true.

§ 5 Design Decisions

An alternative approach is to not have the *movable-box* wrapper. Instead, we can just constrain `single_view` to only accept `movable T`. However, this is inconsistent with the rest of the range library.

§ 6 Future Work

copyable-box is not only used in `single_view`, but also used in lots of views that hold a function object. For example, `transform_view` stores the function inside *copyable-box<F>*. I think as view is no longer required to be copyable, there should be no constraint for the function object F being copyable. And in fact, `transform_view` stores a base view: `v_base_`. If `base_` is not copyable, there is no point to have *copyable-box<F>*. This is the case for all other views that store the function objects. Potentially, the *copyable-box* can be removed and replaced by the *movable-box* that is proposed in this paper.

§ 7 References

[P1456R1] Casey Carter. 2019-11-12. Move-only views.
<https://wg21.link/p1456r1>